

正则表达式 (Regular Expression)

正则表达式(Regular Expression)

- 字符串已经具备的能力：
 - **in** 判断子字符串是否出现: '213-867-5309' in 'My phone number : 213-867-5309.我的住址是邯郸路213号86#401'
 - **find**和**index** 判断子字符串出现的位置
 - 能否判断身份证\手机号码？

正则表达式

- 但是如果想要知道字符串里面**是否包括合法的电话号码**，以及这些号码（国家号，区号，电话号）是什么？
- 如：1.从英文小说中查找 hi，但不能把 him，high，history也找到
2.要找后面不远处跟着一个Lucy的 hi
- 字符串并没有现成的方法来调用，需要用**正则表达式**：
可以判断某个**模式(pattern)**的字符串
 - 可以根据模式来**查找**是否在字符串中出现,以及知道模式在字符串出现的位置
 - 可以根据模式来**分割**字符串
 - 可以根据模式来**替换**字符串

正则表达式

- 一种字符串匹配的模式（pattern）
- 通过使用正则表达式，可以：

常用正则表达式

中文字符	双字节字符	空白行	Email地址	网址URL	手机（国内）	电话号码（国内）	负浮点数	匹配整数
正浮点数	腾讯QQ号	邮政编码	IP	身份证号	格式日期	正整数	负整数	用户名

可以使用正则表达式来识别文档中的特定文本，完全删除该文本或者用其他文本替换它。

- 基于模式匹配从字符串中提取子字符串。
可以查找文档内或输入域内特定的文本。
- 测试网址：<https://www.sojson.com/regex/generate>



使用模式操作字符串

- 一、re模块的函数

模式子字符串即正则表达式

- 如：re.search(*patStr*, longStr, flags)
- 如：re.match(*patStr*, longStr, flags)
- 如：re.findall(*patStr*, longStr, flags)
- 如：re.split(*patStr*, longStr, flags)

- 二、正则表达式对象的方法

- regex = re.compile(*patStr*, flags) # 构造正则表达式对象
- 如：regex.search(longStr)
- 如：regex.match(longStr)
- 如：regex.findall(longStr)
- 如：regex.split(longStr)

4.2.2 re模块主要方法

- **re模块**的主要方法列表

- pattern:匹配模式 string：要匹配的字符串 flag：匹配选项

方法	说明
search (pattern,string[,flags])	在字符串中寻找模式 若匹配，返回Match对象（参见4.2.5），否则返回None
match (pattern,string[,flags])	从字符串的 开始处 匹配模式 若匹配，返回Match对象，否则返回None
findall (pattern,string[,flags])	列出字符串中模式的所有匹配项 返回匹配结果列表，若pattern含有组(子模式)，同时返回组的列表
split (pattern,string[,maxsplit=0])	根据模式匹配项（匹配分割符）分割字符串，返回分割后的字符串列表，maxsplit为分割的最大次数
sub (pat,repl,string[,count=0])	将字符串中所有pat的匹配项用repl替换；并返回替换后的字符串，count为替换的最大次数
subn (pat,repl,string[,count=0])	将字符串中所有pat的匹配项用repl替换；并返回元组：(替换后的字符串,替换次数)，count为替换的最大次数

• 示例：匹配搜索、分割

+ 练习：在Python中验证表中的匹配结果

```
>>> import re
```

Python语句	匹配结果
<code>re.search('to', "To be,\nor not to be")</code>	<code><_sre.SRE_Match object; span=(14, 16), match='to'></code>
<code>re.match('done quit', 'done') #匹配成功</code>	<code><_sre.SRE_Match object; span=(0, 4), match='done'></code>
<code>print(re.match('done quit','have done')) #不成功</code>	<code>None</code>
<code>print(re.match('to', "To be,\nor not to be"))</code>	<code>None</code>
<code>text = 'alpha. beta....gamma delta'</code> <code>pat = '[a-zA-Z]+'</code> <code>re.findall(pat,text) #查找所有单词</code>	<code>['alpha', 'beta', 'gamma', 'delta']</code>
<code>re.split('[\.]+', text)</code>	<code>['alpha', 'beta', 'gamma', 'delta']</code>
<code>re.split('[\.]+', text, maxsplit=2)</code>	<code>['alpha', 'beta', 'gamma delta']</code>
<code>re.split('[\.]+', text, maxsplit=1) #分割1次</code>	<code>['alpha', 'beta....gamma delta']</code>

re模块:: findall

- **findall(pattern,string[,flags]):** 寻找字符串string中所有与模式pattern匹配的匹配项，以列表的形式返回。如果pattern含有组(子模式)，则仅包含子模式的匹配结果，如果有多个子模式，则匹配结果以tuple形式描述。
- **finditer(pattern,string[,flags]):** 返回一个迭代器，每次返回匹配的下一个match对象

```
matches = re.findall('fo','The quick brown fox jumped for food')
print(matches)
# ['fo', 'fo', 'fo']
```

正则表达式组成

如：1.从英文小说中查找 *hi*，但不能把*him*，*high*，*history*也找到

正则表达式：**\bhi\b**

2.要找后面不远处跟着一个*Lucy*的 *hi*

正则表达式：**\bhi\b.*\bLucy\b**

元字符

普通字符

- **正则表达式**就是用于描述这些规则（模式）的语言
- **正则表达式**是普通字符（包括转义字符）、特殊字符（称为**元字符**）及其不同组合来构成的

4.2.1 正则表达式元字符

如：\b . * + () 都是元字符

正则表达式元字符分类：

- 字符类
- 预定义字符类
- 边界匹配符
- 重复限定符
- 分组符()
- 选择符|

4.2.1 正则表达式元字符-字符类

元字符	说明
[]	匹配位于[]中的任意一个字符
-	用在[]之内用来表示范围-可以在多个位置表示范围
^	用在[]之内用来表示否定^只在第一个位置起作用

- **字符类**：由一对[]方括号括起来的字符集合，定义方式如下：
 - **[xyz]**:枚举字符集，匹配括号中任意字符
 - **'[pjc]ython'**可以匹配'python'、'jython'、'cython'
 - **[a-z]**:指定范围的字符，匹配指定范围的任意字符。
 - **'[a-zA-Z0-9]'**可以匹配一个任意大小写字母或数字
 - 如果连字符-也要出现在字符类中，可以采用转义，比如 **'[A-Za-z0-9\-_]'**
 - 连字符也可以放在最前面或者最后面，比如 **'[A-Za-z0-9_-]'**
 - **[^xyz]**:否定枚举字符集，匹配不在括号中任意字符
 - **'[^a-z]'**可以匹配非小写英文字母的字符

例：

```
>>> import re
>>> re.findall('[^abc]','^abcd^')
['^', 'd', '^']
>>> re.findall('[ab^c]','^abcd^')
['^', 'a', 'b', 'c', '^']
>>> re.findall('[\^abc]','^abcd^')
['^', 'a', 'b', 'c', '^']
>>> re.findall('[-abc]','^ab-cd^')
['a', 'b', '-', 'c']
>>> re.findall('[abc-]','^ab-cd^')
['a', 'b', '-', 'c']
>>> re.findall('[a-bc]','^ab-cd^')
['a', 'b', 'c']
>>> re.findall('[a\-bc]','^ab-cd^')
['a', 'b', '-', 'c']
```

4.2.1 正则表达式元字符-预定义字符类

- **预定义字符类**：正则表达式将常常用到的一些特定字符类形成了若干预定义字符类

元字符	说明
.	匹配除换行符以外的任意单个字符
\d	匹配数字类字符，对于英文来说(flags=re.A)，相当于[0-9]
\D	与\d含义相反，匹配非数字字符字，对于英文来说，相当于[^0-9]
\w	匹配能组成单词的字符，对英文来说，相当于[a-zA-Z0-9_]
\W	与\w含义相反,匹配非单词字符。
\s	匹配任何空白字符，对于英文来说，相当于[\t\n\r \f\v]
\S	与\s含义相反，匹配非空白类字符，对于英文来说，相当于[^ \t\n\r \f\v]

例：

```
>>> re.findall('\d',"1233444") #相当于[0-9]
['1', '2', '3', '3', '4', '4', '4']
>>> re.findall('\d',"1 2\n3\t\t3444")
['1', '2', '3', '3', '4', '4', '4']
>>> re.findall('\s',"1 2\n3\t\t3444") #相当于[\t\n\r \f\v]
[' ', '\n', '\t', '\t']
>>> re.findall('\S',"1 2\n3\t\t3444") #相当于[^ \t\n\r \f\v]
['1', '2', '3', '3', '4', '4', '4' ]
>>> re.findall('[\da.]',"123ghhg344.a4")
['1', '2', '3', '3', '4', '4', '.', 'a', '4']
>>> re.findall('[\da].',"123ghhg344.a4")
['12', '3g', '34', '4.', 'a4' ]
>>> re.findall('[\da\S]',"123ghhg344\n.a4")
['1', '2', '3', 'g', 'h', 'h', 'g', '3', '4', '4', '.', 'a', '4']
>>> re.findall('[\da\s]',"123ghhg344 \n.a4")
['1', '2', '3', '3', '4', '4', ' ', ' ', '\n', 'a', '4' ]
>>> re.findall('\.',"123ghhg344\t\n.a4")
['1', '2', '3', 'g', 'h', 'h', 'g', '3', '4', '4', '\t', '.', 'a', '4']
```

其他元字符在[]里不起作用，而^只在第一个位置起作用，-可以在多个位置表示范围

转义字符 \

- **元字符** `^-$*+?{}[]\|()`. 在正则表达式中表示特殊的含义，如果其要作为普通字符使用则需要转义
 - 即在**元字符**前面添加`\`，表示后面的元字符没有特殊的含义而是一个普通字符，比如 `\^` 表示普通字符`^`，`\\s`表示普通字符`\s`而不是空格类字符

正则表达式(pattern)	字符串(string)	说明
fo	'The quick brown fox jumps for food'	匹配其中3个含有'fo'的字符串
1+1=2	'1+1=2'	+为元字符，无法匹配
1\\+1=2	"1+1=2"	["1+1=2"]
(note)	'please(note)'	()为元字符，匹配"note"
\\(note\\)	'please(note)'	匹配"(note)"

- 在python中，用来说明匹配模式的正则表达是通过字符串str类型的对象来传递的，而正则表达式使用的转义字符`\`也是str类型使用的转义字符：（比如 `\b`）
 - 在每个**转义字符**前面再加上`\` → `\\b`
 - 采用**原始字符串** → `r'\b'`
 - 采用re模块的`escape`方法： → `re.escape( '\b' )`

更多转义符号的例子：

通过转义符，将-，\作为普通字符

```
>>> re.findall('[a-z]y', 'ay, sy, zy, -y, \y')
['ay', 'sy', 'zy']
>>> re.findall('[a\\-z]y', 'ay, sy, zy, -y, \y')
['ay', 'zy', '-y']
>>> re.findall('[a\\-z]y', 'ay, sy, zy, -y, \y') #\\字符串中仍然是一个\
['ay', 'zy', '-y']
>>> re.findall(r'[a\\-z]y', 'ay, sy, zy, -y, \y') #\\所以一定要写r,但是写了r后, \\在正则表达式中表示是一个单纯的字符\这里是表示从字符\到字符z
['ay', 'sy', 'zy', '\\y']
>>> re.findall(r'[a\\\\-z]y', 'ay, sy, zy, -y, \y') #\\所以要找到\y, -y, 还需要一个\对-转义
['ay', 'zy', '-y', '\\y']
>>> re.findall(r'[-a\\z]y', 'ay, sy, zy, -y, \y') #\\或将-放在前面
['ay', 'zy', '-y', '\\y']
>>> |
```

4.2.1 正则表达式元字符-边界匹配符

- **0宽度边界匹配符**：字符串匹配往往涉及从某个位置开始匹配，例如字符串的开始与结尾，行的开头或结尾、单词边界等，边界匹配符用于匹配字符串的位置，**不会消耗**模式中的字符

元字符	说明
<code>^</code>	表示匹配的模式位于 字符串的开始 ，在 多行模式 下（re.M）匹配 每行的行首 以 <code>^</code> 后面的字符开头的字符串，如： <code>^a</code> 匹配 <code>abc</code> 中的 <code>a</code> ，不匹配 <code>bat</code> 中的 <code>a</code>
<code>\$</code>	表示匹配的模式位于 字符串结尾 。在 多行模式 下匹配 每行的行尾 如： <code>c\$</code> 匹配 <code>abc</code> 中的 <code>c</code> ，不匹配 <code>acb</code> 中的 <code>c</code>
<code>\b</code>	匹配单词头或单词尾，即表示前面或者后面为非单词类字符 如： <code>r\bfoo\b</code> 匹配 <code>foo</code> ， <code>foo.</code> <code>(foo)</code> <code>bar foo baz</code> 但不匹配 <code>foobar</code> <code>foo3</code> 注意 :str中 <code>'\b'</code> 表示退格字符，所以这些与标准转义字符重复的元字符必须使用 两个\\ 字符，在python中也可以使用原始字符串 <code>r""</code> 或 <code>r</code> ，即 <code>\\bfoo\\b</code> 或 <code>r\bfoo\b</code>
<code>\B</code>	与 <code>\b</code> 含义相反，表示位于单词中间 如： <code>py\B</code> 匹配 <code>python</code> <code>py3</code> <code>py2</code> ，但不匹配 <code>happy</code> <code>sleepy</code> <code>py!</code>

例：零宽界定符 zero-width assertions

```
>>> re.findall('^s',"sad see asdd")
```

```
['s']
```

```
>>> re.findall(r'\Bpy','sleepy" "py!') #匹配不在单词开始的py
```

```
['py']
```

```
>>> re.search(r'\Bpy','sleepy" "py!')
```

```
<_sre.SRE_Match object; span=(4, 6), match='py'>
```

```
>>> re.findall(r'py\B','sleepy" "py!') #匹配不在结尾的py
```

```
[]
```

```
>>> re.findall(r'py\B','sleepy" "py!happybirthday')
```

```
['py']
```

```
>>> re.search(r'py\B','sleepy" "py!happybirthday')
```

```
<_sre.SRE_Match object; span=(15, 17), match='py'>
```

例：

- **re.M(指定多行模式)**: 使用 "^" 只匹配字符串的开始，而 \$ 则只匹配字符串的结尾。当本标志指定後， "^" 匹配字符串的开始和字符串中每行的开始。同样的， \$ 元字符匹配字符串结尾和字符串中每行的结尾（直接在每个换行之前）

```
>>> text = r"whe King and Queen of Hearts were seated on their throne when they arrived,  
with a great crowd assembled about them--all sorts of little birds and beasts,  
ws well as the whole pack of cards: the Knave was standing before them, in  
whains with a soldier on each side to guard him; and near the King was the  
White Rabbit, with a trumpet in one hand, and a scroll of parchment in the  
other. In the very middle of the court was a table, with a large dish of tarts  
upon it: they looked so good, that it made Alice quite hungry to look at them--  
'I wish they'd get the trial done,' she thought, 'and hand round the  
refreshments!'"
```

```
>>> re.findall('^w',text)
```

```
['w']
```

```
>>> re.findall('^w',text,re.M)
```

```
['w', 'w', 'w', 'w']
```

```
>>> re.findall('^w\\w+',text,re.M)
```

```
['whe', 'with', 'ws', 'whains']
```

例：

注意:str中'\b'表示退格字符，所以这些与标准转义字符重复的元字符必须使用两个\\字符，在python中也可以使用原始字符串r"""或r"，即"\\bfoo\\b" 或 r"bfoo\b"

- 再看转义符

```
>>> re.findall('\bo\b',"fofo ofo oof\bo\b")
```

```
['\x08o\x08']
```

```
>>> re.findall(r'\bo\b',"fofo ofo oof\b o\b")
```

```
['o']
```

```
>>> re.search(r'\bo\b',"fofo ofo oof\bo\b")
```

```
<re.Match object; span=(13, 14), match='o'>
```

```
>>> re.findall(r'\bo\b',"fofo ofo oof\b o\b o")
```

```
['o', 'o']
```

4.2.1 正则表达式元字符-重复限定符

- **重复限定符**：指定重复的次数

正则表达式	说明
$X\{n,m\}$	X 重复 n 到 m 次 如： <code>"o{1,3}"</code> 匹配 <code>"fooo<u>ooo</u>d"</code> 中的前3个 <code>"o"</code> 和后3个 <code>"o"</code> <code>['ooo', 'ooo']</code>
$X\{n,\}$	X 至少重复 n 次 如： <code>"o{2,}"</code> 匹配 <code>"fooooooood"</code> 中的所有的 <code>"o"</code> ，不匹配 <code>"bob"</code> 中的 <code>"o"</code>
$X\{n\}$	X 重复 n 次 如： <code>r"\b[0-9]{3}"</code> 匹配 <code>"000"</code> ~ <code>"999"</code> ， <code>"o{2}"</code> 匹配 <code>"food"</code> 中的两个 <code>"o"</code> ，不匹配 <code>"bob"</code> 中的 <code>"o"</code>
X^+	X 重复1次或多次，等价于 $X\{1,\}$ 如： <code>'zo+'</code> 匹配 <code>"zo"</code> ， <code>"zoo"</code> ，但不匹配 <code>"z"</code>
X^*	X 重复0次或多次，等价于 $X\{0,\}$ 如： <code>'zo*'</code> 匹配 <code>"zo"</code> ， <code>"zoo"</code> ， <code>"z"</code>
$X?$	X 字符可选，即 X 重复0次或1次，等价于 $X\{0,1\}$ 如： <code>'colou?r'</code> 匹配 <code>"color"</code> ， <code>"colour"</code> ；

例：

```
>>> re.findall('o{1,3}', "fofofoooof")
```

```
['o', 'oo', 'ooo']
```

```
>>> re.findall('o{1,3}', "fofoooooof")
```

```
['o', 'ooo', 'oo']
```

```
>>> re.findall('o{2,}', "fofoooooof")
```

```
['ooooo']
```

```
>>> re.findall('\d{3}', "123 333 5567777")
```

```
['123', '333', '556', '777']
```

匹配算法：贪婪性匹配算法

- Python针对重复限定符，默认采用**贪婪性匹配算法**
- **贪婪性匹配算法**是指重复限定符会导致正则表达式引擎**尽可能多 (leftmost or largest)**地匹配要重复的前导字符，只有当这种重复引起整个正则表达式匹配失败的情况下，引擎会进行回溯

```
>>> import re
>>> re.findall('<.+>', '<book><title>Python</title><author>Dong</author></book>')
['<book><title>Python</title><author>Dong</author></book>']
```

贪婪算法返回了一个最左边的最长匹配！

如何如我们期望的那样匹配XML的开始与结束标签？

匹配算法：懒惰性匹配算法

- 如果在限定符后面加后缀“**?**”，正则表达式引擎则使用**懒惰性匹配算法**
- 懒惰性匹配算法**是指重复限定符会导致正则表达式引擎**尽可能少**地重复前导字符，只有当这种重复引起整个正则表达式**匹配失败**的情况下，引擎会进行回溯

符号	说明
*?	重复任意次，但尽可能少重复
+?	重复1次或更多次，但尽可能少重复
??	重复0次或1次，但尽可能少重复
{n,m}?	重复n到m次，但尽可能少重复
{n,}?	重复n次以上，但尽可能少重复

```
>>> import re
>>> re.findall('<.+?>', '<book><title>Python</title><author>Dong</author></book>')
['<book>', '<title>', '</title>', '<author>', '</author>', '</book>']
>>> re.findall(r'[\d]{3,4}?', '0421') # 匹配什么？
```

```
>>> re.findall(r'[\d]{3,4}', '0421')
['0421']
>>> re.findall(r'[\d]{3,4}?', '0421')
['042']
```

4.2.1 正则表达式元字符

如：`\b . * + ()` 都是元字符

正则表达式元字符分类：

- 字符类
- 预定义字符类
- 边界匹配符
- 重复限定符
- 分组符()
- 选择符|

4.2.1/4.2.5 正则表达式元字符-分组符

- 分组符“**()**”：将要匹配的模式进一步**分组**，也称为**子模式**。
 - **子模式应用一**：
 - **()**包含的子模式内的**内容**作为一个整体出现，可以应用前面介绍的重复限定符，表示子模式重复多次

findall(pattern,string[,flags]): 寻找字符串string中所有与模式pattern匹配的匹配项，以列表的形式返回。

如果pattern含有组(子模式)，则仅包含子模式的匹配结果，如果有多个子模式，则匹配结果以tuple形式描述。

例如‘ (red)+’ 可以匹配 ‘ redred’
‘ redredred ‘ 等多个重复‘ red’ 的情况

```
>>> import re
>>> re.findall(r'(red)+','redredred redcolor redredpython')
['red', 'red', 'red' ]
```

正则表达式	说明
(pattern)?	允许模式重复0次或1次
(pattern)*	允许模式重复0次或多次
(pattern)+	允许模式重复1次或多次
(pattern){m,n}	允许模式重复m~n次

4.2.1/4.2.5 正则表达式元字符-分组符

- 分组符“**()**”：将要匹配的模式进一步分组，也称为子模式。
 - **子模式应用二**：在获得匹配的模式的同时也可以获得各个匹配的子模式。
 - 子模式可以嵌套

```
>>> telNumber = "Suppose my Phone No. is 0535-1234567,
yours is 010-12345678, his is 025-87654321."
>>> re.findall(r'(\d{3,4})-(\d{7,8})',telNumber)
[('0535', '1234567'), ('010', '12345678'), ('025', '87654321')]
>>> re.findall(r'((\d{3,4})-(\d{7,8}))',telNumber)
[('0535-1234567', '0535', '1234567'), ('010-12345678', '010',
'12345678'), ('025-87654321', '025', '87654321')]
```

- `re.findall(pattern,string[,flags])`:
返回匹配结果列表，若pattern含有子模式，则仅输出子模式，**如果有多个子模式，则以tuple的形式描述**

4.2.2 re模块主要方法

- **re模块**的主要方法列表

- pattern:匹配模式 string：要匹配的字符串 flag：匹配选项

方法	说明
search (pattern,string[,flags])	在字符串中寻找模式 若匹配，返回Match对象（参见4.2.5），否则返回None
match (pattern,string[,flags])	从字符串的 开始处 匹配模式 若匹配，返回Match对象，否则返回None
findall (pattern,string[,flags])	列出字符串中模式的所有匹配项 返回匹配结果列表，若pattern含有组(子模式)，同时返回组的列表
split (pattern,string[,maxsplit=0])	根据模式匹配项（匹配分割符）分割字符串，返回分割后的字符串列表，maxsplit为分割的最大次数
sub (pat,repl,string[,count=0])	将字符串中所有pat的匹配项用repl替换；并返回替换后的字符串，count为替换的最大次数
subn (pat,repl,string[,count=0])	将字符串中所有pat的匹配项用repl替换；并返回元组：(替换后的字符串,替换次数)，count为替换的最大次数

4.2.1/4.2.5 正则表达式元字符-分组符

- 每个group有一个编号:

`re.search(r'(\d{3,4})-(\d{7,8})', telNumber)`

- 按照从左到右的左括号的顺序从1开始进行编号；编号0表示整个匹配的模式。
- 可以通过match对象的group和groups函数来获得匹配的group，可以通过span、start、end等函数获得group匹配的位置。传递的参数可以是单个组编号，也可以是多个组编号，此时返回包含各组内容或者位置的tuple。如果不传递参数，缺省为group 0。

Match对象方法	说明
<code>group([group1,...])</code>	返回匹配的1个或多个子模式所对应的内容，缺省为group 0，即表示模式对应的内容
<code>start([group])</code>	返回指定的子模式匹配的子字符串在原字符串的起始位置
<code>end([group])</code>	返回指定子模式内容的结束位置的前一个位置
<code>span([group])</code>	返回指定子模式内容的结束位置的前一个位置
<code>groups()</code>	返回一个包含匹配的所子模式内容的元组
<code>groupdict()</code>	返回包含匹配的所有命名子模式内容的字典

match对象

re模块: search和match对象

- python提供模块re来使用正则表达式, 其中最主要的方法是

`re.search(pattern, string [, flags])`

- 在字符串string中寻找第一个与模式pattern匹配的子字符串。若找到匹配, 则返回一个Match对象, 否则返回None。flags为匹配时的选项
- 最简单的正则表达式就是如下所示的一个简单字符串, string中包含该字符串则匹配

```
<_sre.SRE_Match object; span=(16, 18), match='fo'>
pattern==>fo
found==>fo
fo
```

```
import re
match = re.search('fo', 'The quick brown fox jumped for food')
print(match)
# <_sre.SRE_Match object; span=(16, 18), match='fo'>

print('pattern==>%s\nfound==>%s' % (match.re.pattern, match.group()))
print(match.string[match.start():match.end()])
```

例子

```
>>> import re
```

```
>>> >>> re.search(r'(red)+','redredred redcolor redredpython')
```

```
<_sre.SRE_Match object; span=(0, 9), match='redredred' >
```

```
>>> matchObj = re.search(r'(red)+','redredred redcolor redredpython')
```

```
>>> matchObj.group()
```

```
'redredred'
```

```
>>> matchObj.span()
```

```
(0, 9)
```

```
>>> matchObj.start()
```

```
0
```

```
>>> matchObj.end()
```

```
9
```

```
>>> re.search(r'(red)?','redredred')
```

```
<_sre.SRE_Match object; span=(0, 3), match='red'>
```

```
>>> re.search(r'(red)*','redredred')
```

```
<_sre.SRE_Match object; span=(0, 9), match='redredred'>
```

例子：search和match对象

- 模式可以将其进一步分解成多个子模式，这些子模式以group的方式来描述

```
>>> match1= re.search(r'((\d){2})-)?(\d){2,3})-(\d){7,8}','Tel: 86-21-65642222,')
```

```
>>> match1.group()
```

```
'86-21-65642222'
```

```
>>> match1.group(0)
```

```
'86-21-65642222'
```

```
>>> match1.group(1,2,3,4)
```

```
('86-', '86', '21', '65642222' )
```

```
>>> match1.group(1)
```

```
'86-'
```

```
>>> match1.groups()
```

```
('86-', '86', '21', '65642222' )
```

```
>>> match1.span()
```

```
(5, 19)
```

```
>>> match1.span(1)
```

```
(5, 8)
```

```
>>> match1.span(2)
```

```
(5, 7)
```

```
>>> match1.span(3)
```

```
(8, 10)
```

```
>>> match1.span(4)
```

```
(11, 19)
```

```
>>> match2 = re.search(r'(\d){4})-(\d){1,2})-(\d){1,2}','Today: 2016-10-31,')
```

```
>>> match2.groups()
```

```
('2016', '10', '31')
```

```
>>> match2.group()
```

```
'2016-10-31'
```

```
>>> match2.group(0)
```

```
'2016-10-31'
```

```
>>> match2.span()
```

```
(7, 17)
```

```
>>> match2.span(1),match2.span(2),match2.span(3)
```

```
((7, 11), (12, 14), (15, 17))
```

```
>>> match2.group(1,2,3)
```

```
('2016', '10', '31')
```

练习：search和match对象

```
match2 = re.search(r'(([\d]{2})-)?([\d]{2,3})-([\d]{7,8})','Tel: 21-65642222,')
print(match2)
#<_sre.SRE_Match object; span=(5, 16), match='21-65642222'>
print(match2.groups())
print('group 0:%s,span %s' % (match2.group(0),match2.span()))
print(match2.group(1,2))
for k,v in enumerate(match2.groups(),1):
    print('group %d: %s, span %s' % (k,v,match2.span(k)))
```

```
(None, None, '21', '65642222')
group 0: 21-65642222, span (5, 16)
(None, None)
group 1: None, span (-1, -1)
group 2: None, span (-1, -1)
group 3: 21, span (5, 7)
group 4: 65642222, span (8, 16)
```

findall, search ...More

```
>>> match1 = re.findall(r'(([\d]{2})-)?([\d]{2,3})-([\d]{7,8})','Tel: 86-21-65642222,Tel: 76-31-65642244,')
>>> match1
[('86-', '86', '21', '65642222'), ('76-', '76', '31', '65642244')]
```

```
>>> match2 = re.search(r'(([\d]{2})-)?([\d]{2,3})-([\d]{7,8})','Tel: 86-21-65642222,Tel: 76-31-65642244,')
>>> print(match2)
<_sre.SRE_Match object; span=(5, 19), match='86-21-65642222'>
>>> print(match2.groups())
('86-', '86', '21', '65642222')
>>> match2.group()
'86-21-65642222'
>>> match2.group(1,2,3,4)
('86-', '86', '21', '65642222')
```

re模块:: finditer(1)

- **search**方法仅仅返回第一个与模式匹配的子字符串，要寻找所有的匹配，可采用：
- **finditer(pattern,string[,flags]):** 返回一个迭代器，每次返回匹配的下一个match对象

```
matches_iter = re.finditer('fo','The quick brown fox jumped for food')
for index, match in enumerate(matches_iter,1):
    print('%d: [%d,%d)==>%s'% (index, match.start(), match.end(), match.group()))
```

```
1: [16,18)==>fo
2: [27,29)==>fo
3: [31,33)==>fo
```

```
>>> match3= re.finditer(r'(([\d]{2})-)?([\d]{2,3})-([\d]{7,8})','Tel: 86-21-65642222,Tel: 76-31-65642244,')
>>> for index,match in enumerate(match3,1): #match3是迭代器
    print('%d:[%d,%d)==>%s'% (index, match.start(), match.end(), match.group()))
```

```
1: [5,19)==>86-21-65642222
2: [25,39)==>76-31-65642244
```

4.2.1/4.2.5 正则表达式元字符-分组符(不作要求)

- 有的时候可能要进行分组，但是用户并不关心某个组具体匹配的子模式是什么
- 可以通过在(后面添加?表示对该分组匹配的内容不感兴趣
- 不感兴趣的组也不占用组编号

```
match2 = re.search(r'(?:([\d]{2})-)?([\d]{2,3})-([\d]{7,8})', 'Tel: 21-65642222,')
print(match2.groups())
print('group 0: %s, span %s' % (match2.group(0), match2.span()))
for k, v in enumerate(match2.groups(), 1):
    print('group %d: %s, span %s' % (k, v, match2.span(k)))
```

```
(None, '21', '65642222')
group 0: 21-65642222, span (5, 16)
group 1: None, span (-1, -1)
group 2: 21, span (5, 7)
group 3: 65642222, span (8, 16)
```

4.2.1/4.2.5 正则表达式元字符-分组符(不作要求)

- 可以给分组(子模式)命名, 方便引用, 而且更改正则表达式也不会有影响

- 在(之后添加?P<groupname>来给分组命名

- match对象的groupdict函数返回一个字典, 其key为那些命名组的名字, 值为对应匹配的子模式

- match对象的group,span,start,end等函数可以通过组名来访问, 当然也可以通过组编号来访问

(None, None, '21', '65642222')

{'country': None, 'city': '21', 'phone': '65642222'}

group 0: 21-65642222, span (5, 16)

group country: None, span (-1, -1)

group city: 21, span (5, 7)

group phone: 65642222, span (8, 16)

```
match2 = re.search(r'((?P<country>[\d]{2})-)?(?P<city>[\d]{2,3})-(?P<phone>[\d]{7,8})', 'Tel: 21-65642222,')
print(match2.groups())
print(match2.groupdict())
print('group 0: %s, span %s' % (match2.group(0), match2.span()))
for k in match2.groupdict():
    print('group %s: %s, span %s' % (k, match2.group(k), match2.span(k)))
```

4.2.1/4.2.5 正则表达式元字符-分组符(不作要求)

- 正则表达式还支持分组引用(backreference), 表示要匹配前面分组所匹配的子模式的内容
- \N 表示引用前面的编号为N的分组匹配的内容
 - 只能引用前面已经匹配的编号, 不能引用后面的分组的编号, 也不能引用编号0
- (?P=groupname)表示引用前面名字为groupname的命名分组匹配的内容

```
pattern = r'((\b\w+)\s+\2)' #匹配两个连续的重复单词
print(re.findall(pattern, 'Paris in the the spring')) [('the the', 'the')]
```

```
pattern = r'(\b\w+)\s+\1'
match = re.search(pattern, 'Paris in the the spring')
print(match.group()) the the
print(match.groups()) ('the',)
pattern = r'(?P<word>\b\w+)\s+(?P=word)'
print(re.search(pattern, 'Paris in the the spring').group()) the the
```

4.2.5 子模式与match对象（不作要求）

- 子模式扩展语法（0宽度的子模式扩展自己看）

语法	说明
(? iLmsux)	给出用于该组匹配的匹配选项flag， flag为iLmsux中的一个或者多个字符，参看后面将要介绍的匹配选项
(?:...)	匹配但不捕获（即对该组不感兴趣）该匹配的子表达式
(?P<groupname>)	为子模式命名，方便对于该组的引用
(?P=groupname)	表示在此之前的命名为groupname的子模式
(?=...)	与前面介绍的\b等类似匹配但不消耗字符。用于正则表达式之后，表示如果=后的内容在字符串中出现则匹配，但不会消耗字符串中的内容
(?!...)	与(=...)含义相反，用于正则表达式之后，表示如果!后的内容在字符串中不出现则匹配，但不会消耗字符串中的内容
(?<=...)	用于正则表达式之前，与(=...)含义相同
(?<!...)	用于正则表达式之前，与(?!...)含义相同
(?#...)	表示后面为注释

4.2.1/4.2.5 正则表达式元字符-选择符

- 选择符“|”，用于选择匹配多个可能的模式（注意不一定是单个字符）中的一个
- 选择符“|”的优先级最低，如需要可使用()来限制选择符的作用范围

Python语句

匹配结果

<code>re.findall('red green blue', 'pink rred ,red,green, ggreen and blue')</code>	<code>['red', 'red', 'green', 'green', 'blue']</code>
<code>re.findall(r'\bred green blue\b', 'pink rred ,red, ggreen and blue')</code>	<code>['red', 'green', 'blue']</code>
<code>re.findall(r'\b(red green blue)\b', 'pink rred ,red, ggreen and blue')</code>	<code>['red', 'blue']</code>

+ 例子：电话号码一般形式为“区号-电话号码”，区号为3位或4位，电话号码为6位或8位数字。

```
>>> import re
```

```
>>> re.findall(r'((0\d{2}|0\d{3})-(\d{8}|\d{6}))', '复旦大学总机021-65642222 浙江大学0571-87951111')
```

```
[('021-65642222', '021', '65642222'), ('0571-87951111', '0571', '87951111')]
```

re模块: match方法和match对象

re.match(pattern, string [, flags])

在字符串的**开始处**匹配模式, 若匹配, 返回match对象, 否则返回None

```
>>> re.match( 'done|quit', 'done') #匹配成功
<_sre.SRE_Match object; span=(0, 4), match='done'>
>>> re.match( 'done|quit', 'quit') #匹配成功
<_sre.SRE_Match object; span=(0, 4), match='quit'>
>>> re.match( 'done|quit', 'have done') #不成功, 返回None
>>> re.search('done|quit', 'have done') #匹配成功
<_sre.SRE_Match object; span=(5, 9), match='done'>
>>> |
```

4.2.2 re模块主要方法

flag：匹配选项 (可以使用|进行组合)

匹配选项	说明
re.I re.IGNORECASE	忽略大小写
re.L re.LOCALE	\w \W \b \B \s \S与本地字符集有关
re.U re.UNICODE	\w \W \b \B \s \S的定义基于Unicode
re.A re.ASCII	\w \W \b \B \s \S的定义基于ASCII
re.S re.DOTALL	使元字符.(dot) 也匹配换行符
re.X re.VERBOSE	忽略模式中的空格（这样可以写成多行），并可以使用#注释，提高可读性
re.M re.MULTILINE	多行匹配模式，^ \$可以匹配每行的开始和结尾

```
pattern = r"""
    (?P<first_three>[\d]{3}) # The first three digits...
    -                       # A literal hyphen...
    (?P<last_four>[\d]{4})  # The last four digits...
"""
m = re.search(pattern, '867-5309', re.VERBOSE)
print(m)
```

4.2.2 re模块主要方法：替换sub

`re.sub(pattern,replace,string[,count=0])`

`re.subn(pattern,replace,string[,count=0])` 与sub类似，只是返回一个元组(替换后的字符串,替换次数)

- 正则表达式不仅提供模式匹配功能，也支持将原字符串string中相应的模式pattern替换成其他字符串replace，最后返回一个新的字符串。count为替换的最大次数，缺省为全部替换
- 替换成的字符串replace可以包含组的引用，表示该分组匹配的内容

```
text = '''Beautiful is better than ugly.  
Explicit is better than implicit.  
Simple is better than complex.  
Complex is better than complicated.  
Flat is better than nested.  
Sparse is better than dense.  
Readability counts.'''
```

* is * than ugly.	*Beautiful is *better than ugly.
Explicit is * than implicit.	Explicit is *better than implicit.
Simple is * than complex.	Simple is *better than complex.
Complex is * than complicated.	Complex is *better than complicated.
Flat is * than nested.	Flat is *better than nested.
Sparse is * than dense.	Sparse is *better than dense.
Readability counts.	Readability counts.

将以字母“b”和“B”开头的单词替换为“*”

```
pattern = r'\b(b|B)\w*\b'  
print (re.sub(pattern, '*', text))
```

将第一个以字母“b”和“B”开头的单词替换为“*”

```
print (re.sub(pattern, '*', text, 1))
```

将以字母“b”和“B”开头的单词前面添加（不作要求）*

```
pattern = r'\b((b|B)\w*)\b'  
print (re.sub(pattern, r'*\1', text))
```

4.2.2 re模块主要方法：分割split

`split(pattern,string[,maxsplit=0])`

- 根据pattern分割字符串string，返回分割后的字符串列表，`maxsplit`为分割的最大次数，缺省为尽最大可能分割

```
text = 'alpha. beta....gamma delta'
print(re.split(r'[\. ]+',text))
print(re.split(r'[\. ]+',text,maxsplit=1))
```

`['alpha', 'beta', 'gamma', 'delta']`

`['alpha', 'beta....gamma delta']`

```
text = r'one444two ,
three.four/file\six?seven[eight]nine|ten'
pattern = r'[\s\d,./\\?[\]|]+'
print(re.split(pattern,text))
```

`['one', 'two', 'three', 'four', 'file', 'six', 'seven', 'eight', 'nine', 'ten']`

```
# 删除字符串中重复的空格
text = 'aaa      bb      c d e      fff      '
print(' '.join(text.split()))
print(' '.join(re.split('[\s]+',text.strip())))
print(re.sub('\s+', ' ', text.strip()))
```

`aaa bb c d e fff`

4.2.4 使用正则表达式对象

- 使用re模块的compile()方法将正则表达式编译生成正则表达式对象，然后再使用正则表达式对象提供的方法 `regex=re.compile(pattern[,flags])` #编译生成正则表达式
- 正则表达式对象的搜索类方法允许传递参数pos和endpos，指明搜索开始和结束位置
- 需要多次调用re方法时可使用正则表达式对象。re模块的方法实际上在内部首先调用re.compile然后使用该正则表达式对象的相应方法

方法	说明
<code>regex.search(string[, pos[, endpos]])</code>	在整个字符串或指定范围搜索；若匹配返回Match对象，否则返回None
<code>regex.match(string[, pos[, endpos]])</code>	与search类似，只是要求模式出现在字符串开头或指定位置开头
<code>regex.finditer(string[, pos[, endpos]])</code>	返回一个在整个字符串或指定范围搜索的迭代器，每次返回匹配的下一个match对象
<code>regex.findall(string[, pos[, endpos]])</code>	在字符串中查找与模式匹配的字符串，返回匹配结果列表。若pattern含有组，则仅返回包含子模式的匹配结果，如果有多个子模式，则匹配结果以tuple形式描述
<code>regex.sub(repl,string[,count=0])</code>	将字符串中所有pattern的匹配项用repl替换；并返回替换后的字符串，count为替换的最大次数
<code>regex.subn(repl,string[,count=0])</code>	将字符串中所有pat的匹配项用repl替换，并返回元组：(替换后的字符串,替换次数)，count为替换的最大次数
<code>regex.split(string[, maxsplit = 0])</code>	根据模式匹配项（匹配分割符）分割字符串，返回分割后的字符串列表，maxsplit为分割的最大次数

4.2.5 使用正则表达式对象

- 示例
phone.py

正则表达式对象
的使用例子
search从index
开始
match object的
group/start/en
d/span方法

```
import re
telNumber = '''Suppose my Phone number is 010-12345678, his is 025-87654321'''
pattern = re.compile(r'(\d{3,4})-(\d{3,4})-(\d{4,8})')
index = 0
while True:
    matchResult = pattern.search(telNumber, index)
    if not matchResult:
        break
    print('-'*30)
    print('Success:')
    for i in range(3):
        print('Searched content:', matchResult.group(i), '\n'
              'Start from:', matchResult.start(i), 'End'
              'at:', matchResult.end(i), '\n'
              'Its span is:', matchResult.span(i))
    index = matchResult.end(2) # or end()
```

Success:
Searched content: 0535-1234567 Start from: 24 End at: 36 Its span is: (24, 36)
Searched content: 0535 Start from: 24 End at: 28 Its span is: (24, 28)
Searched content: 1234567 Start from: 29 End at: 36 Its span is: (29, 36)
Success:
Searched content: 010-12345678 Start from: 47 End at: 59 Its span is: (47, 59)
Searched content: 010 Start from: 47 End at: 50 Its span is: (47, 50)
Searched content: 12345678 Start from: 51 End at: 59 Its span is: (51, 59)
Success:
Searched content: 025-87654321 Start from: 68 End at: 80 Its span is: (68, 80)
Searched content: 025 Start from: 68 End at: 71 Its span is: (68, 71)
Searched content: 87654321 Start from: 72 End at: 80 Its span is: (72, 80)

+ 练习：请在Python中运行该程序

4.2.5使用正则表达式对象

- 可以采用finditer方法返回一个迭代器

```
import re
telNumber = '''Suppose my Phone No. is 0535-1234567,
yours is 010-12345678, his is 025-87654321.'''
pattern = re.compile(r'(\d{3,4})-(\d{7,8})')
for matchResult in pattern.finditer(telNumber):
    print ('-'*30)
    print ('Success:')
    for i in range(3):
        print ('Searched content:', matchResult.group(i),\
            ' Start from:',matchResult.start(i),'End at:',matchResult.end(i),\
            ' Its span is:', matchResult.span(i))
```

+ 练习：请在Python中运行该程序